PROBABILITY & STATISTICS PROJECT

TASK# 01

Collect the data of Dependent variable (CGPA) and Independent Variables (Gender , Age , Attendance_Pct , Study_Hours_Per_Day , Sleep_Hours , Social_Hours_Week , Previous_CGPA).

Collect the data of variables from some authentic Source like WDI, IFS, PBS, SBP, Kaggle, etc.

SOURCE:

https://www.kaggle.com/datasets/robiulhasanjisan/university-student-performance-and-habits-dataset

Techniques Used:

Multiple Linear Regression, Data Cleaning , Data Visualization (Box Plot, Scatter Plot) , Model Evaluation (MSE).

```
# Importing the 'drive' module from the 'google.colab' library.
# This module allows us to access Google Drive within the Colab environment.
from google.colab import drive

# Mounting Google Drive to the Colab environment.
# This step makes your Google Drive accessible from Colab so you can read/write files stored on Google Drive.
# '/content/drive' is the directory where your Google Drive will be mounted in the Colab environment.
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=Tr
```

```
# Importing the necessary library (pandas) for data manipulation and summary statistics
import pandas as pd

# Path of the dataset
file_path = '/content/drive/MyDrive/Probability_DataSet/Student_data.csv'

# Load the dataset into a pandas DataFrame
df = pd.read_csv(file_path)

# Display the first few rows to confirm it's loaded correctly
df.head()
```

	Student_ID	Gender	Age	Major	Attendance_Pct	Study_Hours_Per_Day	Previous_CGPA	Sleep_Hours	Social_Hours_Week
0	ID00001	Male	20	Engineering	83.9	4.4	2.65	9.1	8
1	ID00002	Female	24	Business	80.7	4.0	3.58	4.0	4
2	ID00003	Female	20	Mathematics	91.5	3.9	3.29	6.7	4
3	ID00004	Female	23	Engineering	73.9	8.8	3.48	4.0	6
4	ID00005	Male	21	Economics	79.8	2.2	2.66	8.7	6

Next steps: Generate code with df | New interactive sheet

TASK# 02

Discuss Summary Statistics (Mean, Median, Mode, Quartiles) of each variable included in the Model using R/Python output of summary statistics.

```
# Importing the necessary library (pandas) for data manipulation and summary statistics
import pandas as pd

# Display summary statistics for the dataset (mean, median, mode, quartiles)
summary_stats = df.describe()  # This will give the summary for numerical columns (count, mean, std, min, 25%, 50%, 75%, ma

# For mode (since describe doesn't show mode), we can calculate it for each column
mode_values = df.mode().iloc[0]  # Get the first mode value for each column (because mode can have multiple values)
```

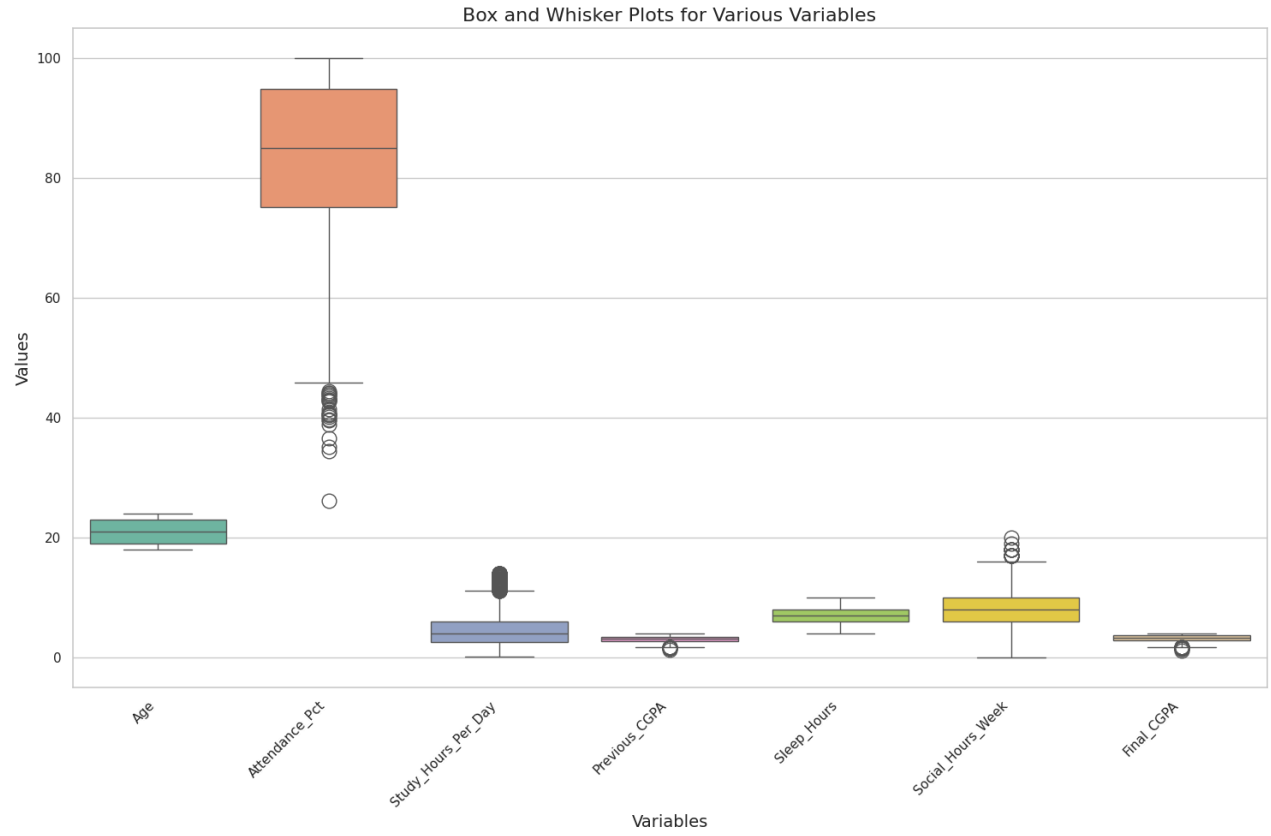
```
# Printing the summary statistics
print("Summary Statistics:")
print(summary_stats)
# Print Extra Line
print(" ")
# Printing the Mode
print("\nMode Values:")
print(mode_values)
```

```
Summary Statistics:
                Age  Attendance_Pct  Study_Hours_Per_Day  Previous_CGPA  \
count  5000.000000     5000.000000          5000.000000    5000.000000
mean     20.948600       83.795740             4.517840       3.093824
std       2.017417       12.867904             2.568104       0.476754
min      18.000000       26.200000             0.100000       1.350000
25%      19.000000       75.100000             2.600000       2.760000
50%      21.000000       85.050000             4.000000       3.100000
75%      23.000000       94.800000             6.000000       3.430000
max      24.000000      100.000000            14.000000       4.000000

       Sleep_Hours  Social_Hours_Week   Final_CGPA
count  5000.000000        5000.000000  5000.000000
mean      7.012060           8.006400     3.272224
std       1.424377           2.797771     0.507954
min       4.000000           0.000000     1.160000
25%       6.000000           6.000000     2.920000
50%       7.000000           8.000000     3.310000
75%       8.000000          10.000000     3.680000
max      10.000000          20.000000     4.000000


Mode Values:
Student_ID               ID00001
Gender                      Male
Age                         19.0
Major                 Psychology
Attendance_Pct             100.0
Study_Hours_Per_Day          2.9
Previous_CGPA                4.0
Sleep_Hours                  7.4
Social_Hours_Week            8.0
Final_CGPA                   4.0
Name: 0, dtype: object
```

Task# 03

Construct Box and Whisker Plots for all the variables in one diagram with different colors and identify outliers if any.

```
# Importing necessary libraries for plotting and visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Set up the style of the plots to make them look better
sns.set(style="whitegrid")

# Create a larger figure to avoid congestion and make it more readable
plt.figure(figsize=(18, 10))  # Increased the figure size for better label visibility

# Create the boxplot for numerical columns
sns.boxplot(data=df[['Age', 'Attendance_Pct', 'Study_Hours_Per_Day', 'Previous_CGPA',
                     'Sleep_Hours', 'Social_Hours_Week', 'Final_CGPA']], palette="Set2",
            flierprops=dict(marker='o', color='red', markersize=12))  # Customizing outlier markers

# Add a title to the plot to describe what it represents
plt.title('Box and Whisker Plots for Various Variables', fontsize=16)

# Labeling the x-axis and y-axis for clarity
plt.xlabel('Variables', fontsize=14)
plt.ylabel('Values', fontsize=14)

# Rotating the x-axis labels for better visibility
plt.xticks(rotation=45, ha='right')  # Adjust rotation and alignment

# Display the plot
plt.show()
```

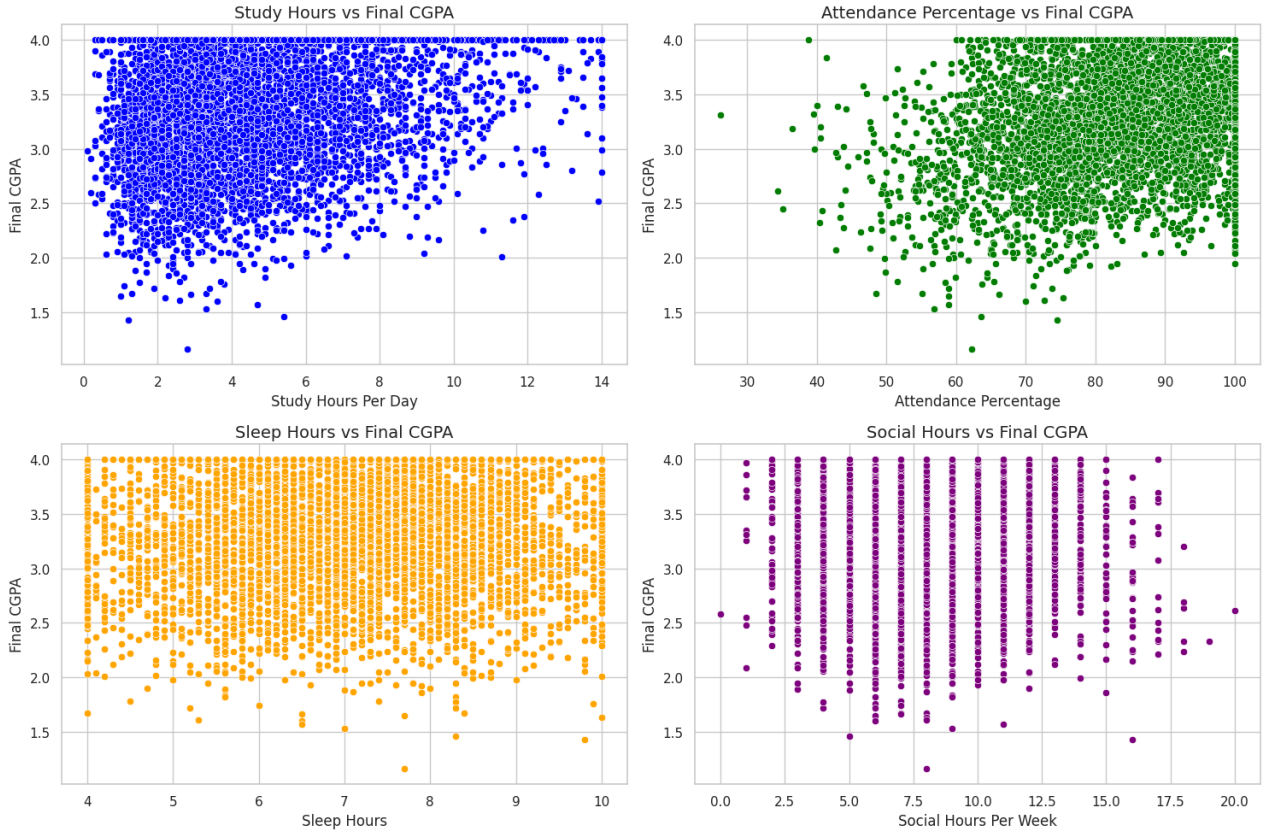


Task# 04

Construct Scatter Plot for all the variables on one grid and interpret each part of the grid separately.

```
# Importing necessary libraries for plotting and visualization
import matplotlib.pyplot as plt  # For creating static, animated, and interactive plots
import seaborn as sns  # For making high-level statistical plots

# Set up the style of the plots to make them look better
sns.set(style="whitegrid")

# Create a scatter plot grid to visualize relationships between different variables
plt.figure(figsize=(15, 10))  # Set the figure size to ensure it's large enough to see all points and labels clearly

# Create a scatter plot to see the relationship between Study Hours and Final CGPA
# Scatter plot is useful to see if there is any correlation between two variables
plt.subplot(2, 2, 1)  # Create a subplot in a 2x2 grid at the first position
sns.scatterplot(x='Study_Hours_Per_Day', y='Final_CGPA', data=df, color='blue')  # Scatter plot between study hours and CGF
plt.title('Study Hours vs Final CGPA', fontsize=14)  # Add title for the subplot
plt.xlabel('Study Hours Per Day')  # Label x-axis
plt.ylabel('Final CGPA')  # Label y-axis

# Create a scatter plot to see the relationship between Attendance and Final CGPA
plt.subplot(2, 2, 2)  # Create the second subplot in the 2x2 grid
sns.scatterplot(x='Attendance_Pct', y='Final_CGPA', data=df, color='green')  # Scatter plot between attendance and CGPA
plt.title('Attendance Percentage vs Final CGPA', fontsize=14)  # Add title for the subplot
plt.xlabel('Attendance Percentage')  # Label x-axis
plt.ylabel('Final CGPA')  # Label y-axis

# Create a scatter plot to see the relationship between Sleep Hours and Final CGPA
plt.subplot(2, 2, 3)  # Create the third subplot in the 2x2 grid
sns.scatterplot(x='Sleep_Hours', y='Final_CGPA', data=df, color='orange')  # Scatter plot between sleep hours and CGPA
plt.title('Sleep Hours vs Final CGPA', fontsize=14)  # Add title for the subplot
plt.xlabel('Sleep Hours')  # Label x-axis
plt.ylabel('Final CGPA')  # Label y-axis

# Create a scatter plot to see the relationship between Social Hours and Final CGPA
```

```
plt.subplot(2, 2, 4)  # Create the fourth subplot in the 2x2 grid
sns.scatterplot(x='Social_Hours_Week', y='Final_CGPA', data=df, color='purple')  # Scatter plot between social hours and CG
plt.title('Social Hours vs Final CGPA', fontsize=14)  # Add title for the subplot
plt.xlabel('Social Hours Per Week')  # Label x-axis
plt.ylabel('Final CGPA')  # Label y-axis

# Automatically adjust the layout to prevent overlap of labels
plt.tight_layout()  # Ensures that the subplots do not overlap and are displayed neatly

# Display the scatter plot grid
plt.show()
```



Task# 05

There should be at least five independent variables (at least 50 observations) in the model. Run two different models including

1. Multiple Linear Regression Model
2. Machine Learning Model

Multiple Linear Regression Model

```
# Import necessary libraries for Linear Regression
from sklearn.linear_model import LinearRegression  # For building linear regression models
from sklearn.model_selection import train_test_split  # For splitting data into training and test sets
from sklearn.metrics import mean_squared_error, r2_score  # For evaluating the model performance

# Select the independent variables (features) and dependent variable (target)
X = df[['Study_Hours_Per_Day', 'Attendance_Pct', 'Sleep_Hours', 'Social_Hours_Week', 'Previous_CGPA']]  # Independent varia
y = df['Final_CGPA']  # Dependent variable (target)

# Split the data into training and test sets (80% for training, 20% for testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize and train the Linear Regression model
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)  # Fit the model using the training data
```

```
# Predict the CGPA values on the test data
y_pred = linear_model.predict(X_test)

# Evaluate the model using Mean Squared Error (MSE) and R-squared
mse = mean_squared_error(y_test, y_pred)  # Mean Squared Error to measure prediction accuracy
r2 = r2_score(y_test, y_pred)  # R-squared to check how well the model fits the data

# Print the evaluation metrics
print("Linear Regression Model Evaluation:")
print(f"Mean Squared Error (MSE): {mse}")
print(f"R-squared: {r2}")
```

```
Linear Regression Model Evaluation:
Mean Squared Error (MSE): 0.026013138313700883
R-squared: 0.9092883364955272
```

Machine Learning Model (Random Forest Regressor)

```
# Import necessary libraries for Random Forest Regressor
from sklearn.ensemble import RandomForestRegressor  # For building a Random Forest model

# Initialize and train the Random Forest model
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)  # 100 trees in the forest
rf_model.fit(X_train, y_train)  # Fit the model using the training data

# Predict the CGPA values on the test data
y_pred_rf = rf_model.predict(X_test)

# Evaluate the model using Mean Squared Error (MSE) and R-squared
mse_rf = mean_squared_error(y_test, y_pred_rf)  # MSE for Random Forest model
r2_rf = r2_score(y_test, y_pred_rf)  # R-squared for Random Forest model

# Print the evaluation metrics for Random Forest
print("Random Forest Model Evaluation:")
print(f"Mean Squared Error (MSE): {mse_rf}")
print(f"R-squared: {r2_rf}")
```

```
Random Forest Model Evaluation:
Mean Squared Error (MSE): 0.01634154564
R-squared: 0.9430146116411523
```

Conclusion

In conclusion section compare the predicted values of the models with original values using graphs and MSE.

```
# Printing a conclusion summary comparing both models' performance

print("Conclusion:")

# Linear Regression Model Evaluation Summary
print("\nLinear Regression Model Evaluation Summary:")
print(f"Mean Squared Error (MSE): {mse}")  # MSE tells us the average squared difference between predicted and actual values
print(f"R-squared: {r2}")  # R-squared shows how well the model fits the data (the higher, the better).

# Random Forest Model Evaluation Summary
print("\nRandom Forest Model Evaluation Summary:")
print(f"Mean Squared Error (MSE): {mse_rf}")  # MSE for the Random Forest model.
print(f"R-squared: {r2_rf}")  # R-squared for the Random Forest model.

# Final evaluation: which model performed better?
if mse < mse_rf:
    print("\nThe Linear Regression model performed better with a lower Mean Squared Error (MSE).")
else:
    print("\nThe Random Forest model performed better with a lower Mean Squared Error (MSE).")

if r2 > r2_rf:
    print("The Linear Regression model explained more variance in the data (higher R-squared).")
else:
    print("The Random Forest model explained more variance in the data (higher R-squared).")
```

```
Conclusion:

Linear Regression Model Evaluation Summary:
Mean Squared Error (MSE): 0.026013138313700883
R-squared: 0.9092883364955272

Random Forest Model Evaluation Summary:
Mean Squared Error (MSE): 0.01634154564
```

```
R-squared: 0.9430146116411523

The Random Forest model performed better with a lower Mean Squared Error (MSE).
The Random Forest model explained more variance in the data (higher R-squared).
```